COMPONENT DESIGN · PHASE 5

Ready to run for everyone.

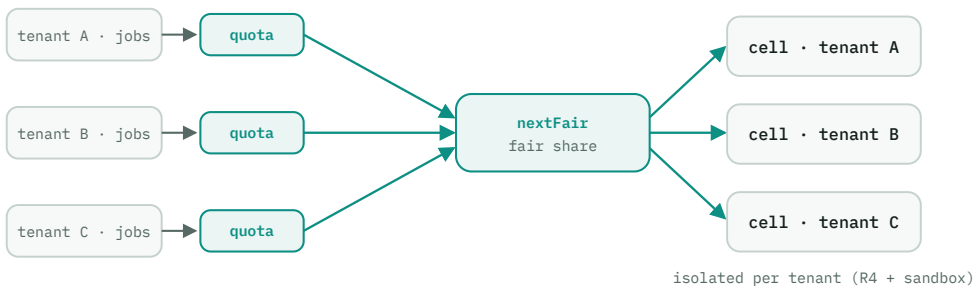
P5 turns a working swarm into a production one — tenant **quotas and fairness**, **disaster recovery**, **SLAs**, and hardware **attestation**. Each is a new pure policy decision plus a hardened shell; and because the security-critical logic stays pure, the security audit is tractable.

01 Production is policy, then hardening

Every production concern splits the same way it always has: a **pure policy decision** (is this tenant over quota? whose turn is it? is this SLO breached? does this attestation verify?) and a **hardened shell** that measures inputs and carries the decision out. Nothing here changes a driver `step` or a verdict — P5 adds policy at the edges and toughens the operational surface.

PRODUCTION CONCERN	PURE DECISION (CORE)	SHELL HARDENING
Multi-tenancy	withinQuota · nextFair	usage tracking · enforcement
Disaster recovery	recoveryPlan · rpoMet	backup/restore · failover · drills
SLAs	evaluate · shouldAlert	metric feed · alert pipeline
Attestation	verifyAttestation	SPIRE TPM plugin

02 Components



Quota gates, then a fair pick, then isolated cells. Each tenant's jobs clear their own quota, a fair-share core chooses whose job runs next, and work lands in tenant-scoped cells. The two decisions — within-quota and fair-next — are pure, so fairness and limits are proven in tests, not observed in production.

Quotas & fairness TENANCY · NEW

Responsibility — keep tenants within limits and share capacity fairly, each tenant's work isolated. Quota and fair-share are pure; usage tracking and enforcement are shell; isolation reuses R4 (dedicated cells) + the sandbox.

◆ FUNCTIONAL CORE · PURE

```
func withinQuota(t Tenant, u Usage, q Quota) bool
func nextFair(pending []Job,
              u map[Tenant]Usage) JobID
  // dominant-resource fair share
func scope(job Job) TenantScope   // isolation
```

▟ IMPERATIVE SHELL · I/O

- track per-tenant usage; reject / queue over-quota jobs
- dispatch the fair pick under the tenant's scope

Boundary — tenant + usage + quota in; a boolean / a chosen job / a scope out. "Two equal-weight tenants, one flooding → the fair pick alternates" is a pure test — fairness proven without a live workload.

Disaster recovery RESILIENCE · NEW

Responsibility — recover from the loss of a whole region or store: decide the recovery plan (failover, re-home, restore from backup) and prove it in drills. The plan is pure; backup/restore and failover are shell.

◆ FUNCTIONAL CORE · PURE

```
func recoveryPlan(loss Loss, fleet FleetState) []Step
  // Step = ReHome{region} | RestoreRegistry{backup}
  //      | Reroute{traffic}
func rpoMet(lastBackup, now Instant, rpo Duration) bool
```

▟ IMPERATIVE SHELL · I/O

- execute steps: re-home agents (P1 failover), restore the FDB registry, redirect the global router
- run DR drills on a schedule

Boundary — a loss event + fleet state in; a recovery plan out. Because the plan is pure, a drill asserts "us-east lost → these exact steps" *before* anything executes.

> **Chaos testing is the shell that validates the plan.** The chaos harness injects the faults — kill cells, partition regions, drop packets — and checks the fleet converges to what `recoveryPlan` said it should. FCIS gives the harness a free oracle: the expected recovery is a pure function, so "reality vs. the core's decision" is the assertion.

SLAs & alerting

OPERATIONS · NEW

Responsibility — turn the P4 metric rollups into SLO verdicts and page only when it matters. Evaluation and alert-worthiness are pure; feeding metrics and firing alerts are shell.

◆ FUNCTIONAL CORE · PURE

```
func evaluate(slo SLO, w Metrics) SLState
  // SLState = Met | AtRisk | Breached
func errorBudget(slo SLO, w Metrics) float
func shouldAlert(cur, prev SLState, h Hysteresis) bool
```

▟ IMPERATIVE SHELL · I/O

- feed the rolled-up metrics window
- fire alerts to the notification pipeline; page on-call

Boundary — an SLO + a metrics window in; a state / a budget / an alert decision out. Hysteresis lives in the pure `shouldAlert`, so "flapping at the threshold doesn't page twice" is a unit test — no accidental alert storms.

Core-tier attestation SECURITY · DELTA

Responsibility — on the core tier, bind identity to real hardware (brief S4) by verifying a TPM/enclave quote. Verification is pure crypto; obtaining the quote is shell. The open tier still relies on quorum + reputation.

◆ FUNCTIONAL CORE · PURE

```
func verifyAttestation(q AttQuote, p AttPolicy) AttResult
  // Valid{measurements} | Invalid
func trustFromAttestation(r AttResult) TrustTier
```

▟ IMPERATIVE SHELL · I/O

- SPIRE's node-attestation plugin fetches the TPM quote at enrollment
- set the issued identity's trust tier from the result

Boundary — a quote + policy in; a verdict / trust tier out. Verifying a quote against expected measurements is a pure, auditable function — attestation raises assurance on the core tier without touching the open tier's model.

03 The security audit is tractable because the cores are pure

P5's security audit reviews the security-critical logic — the quorum `verdict`, `reputation`, `admitOpen`, `verifyAttestation`, `verifySignature`, and the quota checks — and every one of them is a **pure function with property tests**. An auditor can answer "can a minority flip a verdict?" or "can an over-quota tenant sneak a job through?" by reading one function and its tests, not by tracing logic tangled through network and disk I/O. Drawing the FCIS line well in P0 is what makes the phase-5 audit a bounded, readable exercise instead of a spelunk.

04 What P5 defers

P5 makes the swarm safe to run for many tenants, survivable, and accountable to its promises. One phase remains. **P6 (self-tuning + full-scale GA)** writes the last cores — mitosis that splits on a *measured* signal rather than a count (the brief §03 upgrade) and capability/locality placement refinement (short of a solver, per §18) — and validates a sustained run at ~1M nodes. After that, the design is realized: every decision pure, every effect in a shell, from a two-machine start to a million.